POLITECHNIKA PRZYKŁADOWA

Wydział Informatyki i Nauki o Danych
Katedra Sztucznej Inteligencji i Systemów Decyzyjnych

Automatyczna weryfikacja poprawności językowej dokumentów akademickich z zastosowaniem metod przetwarzania języka naturalnego

Praca dyplomowa magisterska

Autor:	mgr inż. Jan Kowalski
Promotor:	dr hab. inż. Anna Nowak, prof. PP
Numer albumu:	123456
Kierunek:	Informatyka
Specjalność:	Sztuczna Inteligencja

Poznań, 2025

Praca napisana pod kierunkiem dr hab. inż. Anny Nowak

Spis skrótów i oznaczeń

NLP – Przetwarzanie Języka Naturalnego (Natural Language Processing)
ML – Uczenie Maszynowe (Machine Learning)
SVM – Maszyna Wektorów Nośnych (Support Vector Machine)
API – Interfejs Programowania Aplikacji (Application Programming Interface)
PDF – Przenośny Format Dokumentów (Portable Document Format)
NER – Rozpoznawanie Encji Nazwanych (Named Entity Recognition)
PoS – Część Mowy (Part of Speech)
F1 – Miara F1, średnia harmoniczna precyzji i czułości (F1 Score)
TP – Prawdziwie Pozytywne (True Positive)
FP – Fałszywie Pozytywne (False Positive)
FN – Fałszywie Negatywne (False Negative)
HMM – Ukryte Modele Markowa (Hidden Markov Model)

Streszczenie

Niniejsza praca opisuje projekt i implementację systemu automatycznej analizy językowej dokumentów akademickich, ze szczególnym uwzględnieniem prac dyplomowych. System wykrywa błędy ortograficzne, interpunkcyjne oraz stylistyczne, a także weryfikuje poprawność użycia skrótów i spójność list wyliczeniowych w dokumentach napisanych w języku polskim i angielskim. W pracy przedstawiono architekturę modułową systemu, omówiono zastosowane metody przetwarzania języka naturalnego oraz zaprezentowano wyniki eksperymentów ewaluacyjnych. System osiąga wartości F1 powyżej 0,85 dla wszystkich analizowanych kategorii błędów, co potwierdza jego przydatność w procesie korekty prac dyplomowych.

Słowa kluczowe: przetwarzanie języka naturalnego, weryfikacja tekstu, analiza lingwistyczna, praca dyplomowa, spaCy, LanguageTool

Abstract

This thesis presents the design and implementation of an automated linguistic analysis system for academic documents, with a focus on diploma theses. The system detects spelling, punctuation, and stylistic errors, and verifies the correct use of abbreviations as well as the coherence of enumeration lists in documents written in Polish and English. The thesis describes the modular architecture of the system, discusses the applied natural language processing methods, and presents the results of evaluation experiments. The system achieves F1 scores above 0.85 for all analysed error categories, confirming its usefulness in the process of reviewing diploma theses.

Keywords: natural language processing, text verification, linguistic analysis, diploma thesis, spaCy, LanguageTool

Chapter 1

Wstęp

1.1 Kontekst i motywacja

Prace dyplomowe stanowią szczytowe osiągnięcie kształcenia akademickiego na poziomie licencjackim i magisterskim. Ich jakość zależy nie tylko od merytorycznej zawartości, lecz również od poprawności językowej, stylistycznej oraz zgodności z obowiązującymi konwencjami edytorskimi. Tradycyjny proces korekty pracy dyplomowej jest czasochłonny i wymaga zaangażowania promotora oraz lektora językowego, co generuje znaczące koszty i wydłuża czas przygotowania dokumentu do oceny.

Automatyczne narzędzia wspomagania redakcji tekstu mogą znacząco usprawnić ten proces. Systemy oparte na metodach ML umożliwiają skuteczną detekcję szerokiego zakresu błędów językowych bez konieczności ręcznej weryfikacji całego dokumentu. Jednocześnie specyfika języka polskiego – bogata fleksja, swobodny szyk wyrazów w zdaniu oraz liczne wyjątki od reguł ortograficznych – sprawia, że projektowanie takich systemów jest znacznie trudniejsze niż dla języka angielskiego.

Wstępna analiza 100 prac dyplomowych złożonych na Wydziale Informatyki w latach 2020 - 2024 wykazała, że najczęstszymi kategoriami błędów są: błędy ortograficzne i literówki (obecne w 78% prac), nieprawidłowe użycie skrótów bez definicji (61%), niespójne listy wyliczeniowe (55%) oraz stosowanie form osobowych zamiast bezosobowych (47%). Przeprowadziłem tę wstępną analizę korzystając z ręcznie anotowanego korpusu tekstów, co pozwoliło na precyzyjne oszacowanie częstości poszczególnych typów błędów.

Dokumenty akademickie przesyłane do systemu mają najczęściej format *Portable Document Format (PDF)*, który wymaga odpowiedniego etapu ekstrakcji tekstu przed właściwą analizą językową. Format PDF jest zorientowany na prezentację wizualną, a nie na logiczną strukturę tekstu, co sprawia, że ekstrakcja struktury bloków – akapitów,

nagłówków, list, tabel i opisów skrótów – stanowi istotne wyzwanie techniczne.

Potrzeba automatyzacji procesu korekty jest wyraźna: przy kilku tysiącach prac dyplomowych składanych corocznie na dużych uczelniach, ręczna weryfikacja każdego dokumentu jest praktycznie niemożliwa. System wspomagający korektę może wykonać wstępną weryfikację w ciągu minut, wskazując promotorowi i autorowi najważniejsze obszary wymagające poprawy.

1.2 Cel pracy

Celem niniejszej pracy jest zaprojektowanie, implementacja i ewaluacja systemu automatycznej weryfikacji poprawności językowej prac dyplomowych, obejmującego następujące funkcjonalności:

- detekcję błędów ortograficznych i literówek z uwzględnieniem nazw własnych i terminologii specjalistycznej;
- weryfikację poprawności definicji skrótów przy pierwszym użyciu, w tym wykrywanie braku definicji oraz nieprawidłowej kolejności rozwinięcia i skrótu;
- sprawdzanie spójności list wyliczeniowych pod względem interpunkcji i wielkości liter;
- detekcję nieodpowiedniego użycia form osobowych czasownika;
- wykrywanie błędów w stosowaniu separatorów dziesiętnych;
- analizę poprawności użycia myślników, półpauz i pauz.

System ma obsługiwać dokumenty w języku polskim oraz angielskim, a jego skuteczność ma być oceniana za pomocą standardowych metryk ewaluacji: precyzji (ang. precision), czułości (ang. recall) oraz miary F1.

1.3 Zakres i ograniczenia pracy

Niniejsza praca obejmuje analizę dokumentów tekstowych w formacie PDF. Zakres badań ograniczony jest do języka polskiego i angielskiego, przy czym szczególny nacisk położono na język polski ze względu na jego złożoność morfologiczną. System nie obsługuje dokumentów zawierających wyłącznie obrazy (skany), dokumentów zaszyfrowanych ani dokumentów w językach innych niż wymienione.

Ograniczenia techniczne systemu obejmują brak obsługi dokumentów o niestandardowej strukturze, takich jak formularze wielokolumnowe czy dokumenty z rozbudowanymi tabelami zawierającymi fragmenty tekstowe. Ponadto skuteczność systemu może być

obniżona w przypadku tekstów zawierających duże fragmenty w językach innych niż obsługiwane, na przykład cytaty w języku niemieckim lub rosyjskim.

Warto podkreślić, że system pełni rolę narzędzia wspomagającego – ostateczna decyzja o akceptacji bądź odrzuceniu danej sugestii zawsze należy do użytkownika. Zarówno autorzy pracy, jak i promotorzy, powinni traktować wyniki działania systemu jako punkt wyjścia do pogłębionej weryfikacji, a nie jako autorytatywne orzeczenie o błędzie.

Niniejsza praca nie obejmuje problematyki wykrywania plagiatów, oceny merytorycznej treści dokumentów ani automatycznego generowania poprawek. Problematyka ta, choć pokrewna, wykracza poza przyjęty zakres badań i wymaga odrębnego opracowania.

1.4 Konwencje językowe w polskich pracach dyplomowych

Polskie normy edytorskie dla prac dyplomowych różnią się w kilku istotnych aspektach od norm anglosaskich. Poniżej omówiono najważniejsze konwencje, które zostały uwzględnione w projekcie systemu weryfikacyjnego.

Formy osobowe. W polskich pracach naukowych panuje konwencja stosowania form bezosobowych lub strony biernej. Autorzy powinni unikać zwrotów takich jak „przeprowadziłem badania” czy „moim zdaniem”, zastępując je formami „przeprowadzono badania” lub „w opinii autora”. Konwencja ta jest szczególnie restrykcyjna w pracach magisterskich i doktorskich, nieco łagodniejsza zaś w pracach inżynierskich, gdzie niekiedy dopuszcza się użycie formy pierwszoosobowej liczby mnogiej („przeprowadziliśmy”).

Separatory dziesiętne. W języku polskim jedynym poprawnym separatorem dziesiętnym jest przecinek. Użycie kropki jako separatora dziesiętnego jest błędem typograficznym, choć powszechnym w pracach studentów przyzwyczajonych do środowisk programistycznych, gdzie standardem jest notacja anglosaska. Wyjątek stanowią bloki kodu źródłowego, gdzie notacja anglosaska jest w pełni akceptowana.

Myślniki i półpauzy. Szczegółowe reguły użycia myślników w języku polskim są często ignorowane przez autorów prac dyplomowych. Najczęstszym błędem jest użycie dywizu (znaku łącznika, dostępnego bezpośrednio na klawiaturze) zamiast półpauzy we wtrąceniach i zakresach. Na przykład, poprawny zapis zakresu lat to „2019–2024” (z półpauzą, bez spacji), natomiast „2019-2024” (z dywizem) lub 2019 - 2024 (z dywizem i spacjami) są błędami edytorskimi.

Skróty i akronimy. Polska norma PN-EN ISO 5966 zaleca definiowanie wszystkich skrótów i akronimów przy pierwszym użyciu w tekście, niezależnie od obecności spisu skrótów. Definicja powinna mieć formę: „pełna nazwa (SKRÓT)”, np. *Przetwarzanie*

Języka Naturalnego (NLP). Odwrotna kolejność, tzn. „(SKRÓT) pełna nazwa”, jest błędem edytorskim. Co istotne, pierwsze użycie skrótu w streszczeniu i w tekście głównym traktuje się jako oddzielne pierwsze użycia – skrót zdefiniowany w streszczeniu powinien być ponownie zdefiniowany przy pierwszym użyciu w tekście głównym.

Spójność list wyliczeniowych. Polskie podręczniki redakcji tekstu naukowego zalecają, aby wszystkie elementy listy wyliczeniowej były jednorodne stylistycznie: albo wszystkie są pełnymi zdaniami (zaczynają się od wielkiej litery, kończą się kropką), albo wszystkie są frazami (zaczynają się od małej litery, kończą się przecinkiem lub średnikiem, a ostatni element kończy się kropką). Mieszanie obu stylów jest błędem.

Wymienione konwencje stanowią bezpośrednią podstawę dla reguł zaimplementowanych w poszczególnych modułach systemu. Każda z nich jest trudna do weryfikacji ręcznie w skali całego dokumentu, co czyni automatyzację szczególnie cenną.

1.5 Struktura dokumentu

Praca składa się z siedmiu rozdziałów merytorycznych. Rozdział 1 wprowadza temat pracy, opisuje motywację oraz zakres badań. Rozdział 2 zawiera przegląd literatury dotyczącej metod przetwarzania języka naturalnego oraz istniejących narzędzi weryfikacji tekstu. Rozdział 3 opisuje metodologię zastosowaną w pracy, w tym architekturę systemu i zasady detekcji poszczególnych kategorii błędów. Rozdział 4 omawia szczegóły implementacyjne systemu. Rozdział 5 prezentuje wyniki eksperymentów ewaluacyjnych. Rozdział 6 zawiera dyskusję wyników i analizę ograniczeń systemu. Rozdział 7 podsumowuje pracę i wskazuje kierunki dalszych badań.

Chapter 2

Przegląd literatury

2.1 Metody przetwarzania języka naturalnego

Przetwarzanie Języka Naturalnego (NLP) jest interdyscyplinarną dziedziną nauki łączącą językoznawstwo, informatykę oraz statystykę. Jej celem jest umożliwienie komputerom automatycznego rozumienia, interpretowania i generowania języka używanego przez ludzi. W ciągu ostatnich dekad NLP przeszło gwałtowną ewolucję – od systemów opartych na regułach, przez modele statystyczne, aż po współczesne modele głębokiego uczenia.

Wczesne systemy NLP opierały się na ręcznie tworzonych gramatykach i słownikach. Takie podejście, choć precyzyjne w wąskich domenach, okazało się niewystarczające przy skalowaniu do języka ogólnego. Zbadałem literaturę z lat 1990 - 2010 i stwierdziłem, że przełomowym momentem było wprowadzenie *Ukrytych Modeli Markowa (HMM)* do zadania automatycznego etykietowania części mowy.

Kolejnym istotnym przełomem było wprowadzenie reprezentacji wektorowych słów (ang. word embeddings). Metoda Word2Vec, zaproponowana przez Mikolova i współpracowników w 2013 roku, umożliwiła uchwycenie semantycznych relacji między słowami w przestrzeni wektorowej o wysokiej wymiarowości. W zaproponowanym modelu słowa o zbliżonym znaczeniu są reprezentowane przez bliskie sobie wektory, co umożliwia wykonywanie operacji arytmetycznych na pojęciach semantycznych.

Modele kontekstowe, takie jak ELMo i BERT, zrewolucjonizowały podejście do reprezentacji tekstu, uwzględniając kontekst zdania przy tworzeniu reprezentacji każdego słowa. W odróżnieniu od statycznych embeddingów, modele te generują odmienną reprezentację dla każdego wystąpienia słowa, w zależności od jego otoczenia leksykalnego. Dzięki temu możliwe jest poprawne rozróżnianie słów wieloznacznych, co ma istotne znaczenie w zadaniach analizy językowej.

Współczesne modele językowe oparte na architekturze Transformer osiągają wyniki zbliżone do człowieka w wielu zadaniach rozumienia języka, takich jak odpowiadanie na pytania, klasyfikacja sentymentu czy streszczanie tekstu. W kontekście weryfikacji tekstu akademickiego szczególne znaczenie mają zadania takie jak: znakowanie części mowy (*Part of Speech (PoS)*), analiza zależności składniowych (ang. dependency parsing), (NER) Named Entity Recognition oraz wykrywanie błędów językowych na poziomie morfosyntaktycznym.

Dla języka polskiego dostępne są pretrenowane modele spaCy `pl_core_news_lg` oraz transformerowy `pl_core_news_trf`. Modele te zostały wytrenowane na korpusach zawierających teksty z Wikipedii, prasy oraz literatury naukowej. Ich skuteczność w zadaniu rozpoznawania encji nazwanych dla języka polskiego wynosi około 0.85–0.87 w metryce F1, co jest wynikiem porównywalnym z modelami dla języka angielskiego.

2.2 Narzędzia gramatyczne i korektory tekstu

Spośród dostępnych narzędzi open-source do weryfikacji tekstu na szczególną uwagę zasługuje LanguageTool – wielojęzyczny korektor gramatyczny i stylistyczny oparty na regułach. Narzędzie to obsługuje ponad 30 języków, w tym język polski, i wykrywa szeroki zakres błędów, od prostych literówek po złożone błędy stylistyczne dotyczące poprawności użycia form fleksyjnych.

LanguageTool oparty jest na trzech głównych mechanizmach: regułach opartych na wyrażeniach regularnych, regułach zdefiniowanych w formie wzorców gramatycznych opartych na tokenach oraz słowniku morfologicznym. Dla języka polskiego słownik morfologiczny Morfologik zawiera ponad 2 miliony form wyrazowych, co czyni go jednym z najbardziej rozbudowanych słowników dla tego języka. Anlaizę poprawności reguł LanguageTool dla języka polskiego przeprowadzili Kowalski i Nowak (2022), osiągając precyzję na poziomie 0,82 dla kategorii błędów gramatycznych.

Alternatywnym podejściem jest wykorzystanie biblioteki spaCy, która oferuje pretrenowane modele językowe dla wielu języków. Modele spaCy są zoptymalizowane pod kątem szybkości i mogą być efektywnie uruchamiane w środowiskach produkcyjnych bez dostępu do GPU. Biblioteka ta oferuje funkcjonalności takie jak: tokenizacja, lemmatyzacja, znakowanie PoS, analiza składniowa oraz rozpoznawanie NER. Wersja modelu `pl_core_news_lg` osiąga dokładność tokenizacji na poziomie 99,8% dla standardowych tekstów polskich.

Warto zaznaczyć, że żadne z dostępnych narzędzi open-source nie oferuje kompleksowej obsługi wszystkich kategorii błędów specyficznych dla polskich prac dyplomowych. W szczególności żadne z nich nie weryfikuje spójności list wyliczeniowych ani poprawności

definicji skrótów zgodnie z polskimi normami edytorskimi określonymi w normie PN-EN ISO 5966.

Inne godne uwagi narzędzia to biblioteka `language_detector` oparta na modelu linguapy, która umożliwia automatyczną detekcję języka tekstu na poziomie zdania lub bloku. Ma to kluczowe znaczenie w dokumentach dwujęzycznych, gdzie niektóre fragmenty (np. cytaty z literatury anglojęzycznej) mogą być napisane w języku angielskim, a pozostałe w języku polskim.

2.3 Weryfikacja definicji skrótów

Poprawna praktyka edytorska wymaga, aby każdy skrót był zdefiniowany przy pierwszym użyciu w tekście poprzez podanie jego pełnego rozwinięcia. W języku polskim standardem jest podanie rozwinięcia przed skrótem w nawiasie, np.: „przetwarzanie języka naturalnego (NLP)". Odwrotna kolejność, np. „(NLP) przetwarzanie języka naturalnego", jest powszechnie uznawana za błąd edytorski, choć w starszych pracach inżynierskich spotyka się obie formy.

Automatyczna weryfikacja definicji skrótów wymaga: identyfikacji wszystkich skrótów w tekście, sprawdzenia, czy każdy z nich jest zdefiniowany przy pierwszym użyciu, oraz walidacji formatu tej definicji. Zadanie to jest nietrywialnym problemem NLP, ponieważ granica między skrótami a zwykłymi słowami pisanymi wielkimi literami może być niejednoznaczna.

Istnieje kilka podejść do identyfikacji skrótów. Najprostsze polega na zastosowaniu wyrażeń regularnych do identyfikacji ciągów znaków pisanych wyłącznie wielkimi literami o długości od 2 do 7 znaków. Podejście bardziej zaawansowane wykorzystuje modele uczenia maszynowego (ML) do rozróżniania skrótów od innych słów pisanych wielkimi literami, takich jak nazwy własne, nazwy organizacji czy skróty jednostek miar. W niniejszej pracy zastosowano podejście hybrydowe, łączące wyrażenia regularne z analizą kontekstu morfosyntaktycznego dostarczonego przez bibliotekę spaCy.

Szczególnym przypadkiem jest użycie skrótu w tzw. blocku akronimów (sekcji spisu skrótów) na początku dokumentu. W takiej sytuacji każdy skrót jest tam zdefiniowany, lecz jego pierwsze użycie w tekście głównym powinno nadal zawierać definicję w nawiasie – chyba że dokument explicite wskazuje, że skróty należy sprawdzać tylko względem spisu skrótów.

2.4 Analiza spójności list wyliczeniowych

Spójność list wyliczeniowych jest aspektem jakości tekstu rzadko poruszanym w literaturze NLP, choć ma istotne znaczenie dla oceny poprawności stylistycznej prac dyplomowych. Zgodnie z polskimi normami edytorskimi, wszystkie elementy listy powinny zaczynać się od tej samej wielkości litery oraz kończyć się tym samym znakiem interpunkcyjnym.

Wyróżnia się dwa główne style list wyliczeniowych:

- **Styl zdaniowy** – każdy element jest pełnym zdaniem, zaczynającym się od wielkiej litery i kończącym się kropką;
- **Styl frazowy** – każdy element jest frazą lub wyrażeniem nominalnym, zaczynającym się od małej litery i kończącym się przecinkiem lub średnikiem, z wyjątkiem ostatniego elementu, który kończy się kropką.

Mieszanie obu stylów w obrębie jednej listy jest traktowane jako błąd spójności. Automatyczna detekcja tego błędu wymaga: identyfikacji bloków listowych w dokumencie; analizy morfosyntaktycznej każdego elementu listy; klasyfikacji stylu listy na podstawie dominującego wzorca; oznaczenia elementów niezgodnych z dominującym stylem. Proces ten wymaga uwzględnienia wyjątków, takich jak elementy list będące definicjami (format: „SKRÓT – rozwinięcie") lub elementy zaczynające się od nazwy własnej.

Poniżej przedstawiono przykład niespójnej listy wyliczeniowej, zawierającej mieszane zakończenia i niejednolite użycie wielkich liter (każdy element jest oznaczony jako błąd LIST_COHERENCE):

- analiza morfologiczna tekstu,
- Weryfikacja definicji skrótów,
- sprawdzanie spójności list wyliczeniowych,
- detekcja błędów interpunkcyjnych.

2.5 Analiza statystyczna struktur składniowych

Oprócz weryfikacji poprawności gramatycznej, w pracach dyplomowych istotna jest analiza strukturalna używanych zdań. Polskie normy edytorskie zalecają stosowanie strony biernej lub form bezosobowych w pracach naukowych. Statystyczna analiza proporcji zdań czynnych, biernych i bezosobowych dostarcza cennych informacji o stylu językowym pracy.

W niniejszym systemie każde zdanie akapitu jest klasyfikowane do jednej z trzech kat-

egorii strukturalnych:

1. **Zdanie czynne** (ang. active voice) – zdanie, w którym podmiot wykonuje czynność. Przykład: „System wykrywa błędy ortograficzne.”
2. **Zdanie bierne** (ang. passive voice) – zdanie, w którym podmiot jest obiektem czynności. Wykrywane przez obecność zależności `aux:pass` w drzewie składniowym spaCy.
3. **Zdanie bezosobowe** – zdanie bez podmiotu, np. „Przeprowadzono analizę danych.” lub zdanie zwrotne z„się”, np. „Stosuje się metodę regresji logistycznej.”

Na potrzeby statystyki procentowej wyznaczany jest stosunek zdań biernych do sumy zdań czynnych i biernych (zdania bezosobowe traktowane są osobno). Zgodnie z analizą korpusu prac dyplomowych, w typowej polskiej pracy magisterskiej udział zdań biernych i bezosobowych powinien przekraczać 60%.

Dokonano analizy porównawczej 30 prac dyplomowych w zakresie proporcji form zdaniowych. Wyniki wskazują, że prace ocenione jako „poprawne stylistycznie” przez komisję wydziałową cechują się średnim udziałem zdań biernych wynoszącym 42% oraz zdań bezosobowych wynoszącym 31%, co daje łącznie 73% zdań o stylu akademickim. W pracach ocenionych negatywnie pod względem stylistycznym proporcja ta spadała do 45–52%.

Wyniki statystyczne dla konkretnego dokumentu są raportowane w obiekcie `Analisys_type` jako pola `passive_count`, `active_count`, `impersonal_count` oraz `passive_ratio`. Informacje te są prezentowane użytkownikowi jako podsumowanie statystyczne, nie jako błędy – każde zdanie osobowe jest natomiast flagowane jako indywidualny błąd `PERSONAL_FORM`.

2.6 Zasoby języka polskiego w NLP

Dostępność wysokiej jakości zasobów językowych jest kluczowym czynnikiem determinującym możliwości budowy systemów NLP dla języka polskiego. W ostatnich latach nastąpił znaczący postęp w tej dziedzinie, jednak zasoby polskie wciąż ustępują pod względem liczebności i różnorodności zasobom anglojęzycznym.

Najważniejszym zasobem dla języka polskiego jest *Narodowy Korpus Języka Polskiego (NKJP)*, zawierający ponad 250 milionów słowoform z różnorodnych źródeł tekstowych: literatury, prasy, tekstów internetowych i mówionych. Korpus ten jest anotowany morfosyntaktycznie zgodnie ze standardem "IPIPAN" i stanowi podstawę trenowania większości polskich modeli NLP.

Słownik morfologiczny Morfologik, oparty na zasobach NKJP, zawiera ponad 2 miliony form wyrazowych z przypisanymi znacznikami morfologicznymi. Słownik ten jest podstawowym komponentem LanguageTool-PL i spaCy-PL. Zaimplementwano (powinno być „Zaimplementowano”) mechanizm dynamicznego rozszerzania słownika o nowe wyrazy specjalistyczne na podstawie analizy dokumentów z danej dziedziny.

Treebank składniowy języka polskiego – Składnica – zawiera ponad 8000 zdań z anotacją zależnościową zgodną ze standardem Universal Dependencies. Na jego podstawie wytrenowany został model spaCy `pl_core_news_lg`, osiągający dokładność znakowania PoS na poziomie 97,8% i analizy zależnościowej na poziomie 89,2% dla tekstów standardowych. Dokładność spada jednak znacząco dla tekstów technicznych i specjalistycznych, co jest istotnym ograniczeniem przy analizie prac dyplomowych z dziedzin ścisłych.

Warto wspomnieć o projekcie HerBERT – polskim modelu transformerowym trenowanym na dużym korpusie polskich tekstów. HerBERT osiąga stan wiedzy w wielu zadaniach NLP dla języka polskiego, w tym w rozpoznawaniu NER i analizie sentymentu. W niniejszym systemie nie zastosowano HerBERT ze względu na wysokie wymagania pamięciowe (ok. 1 GB (Gigabytes) tylko dla modelu) i długi czas inferencji bez GPU, lecz jest to obiecujący kierunek dla przyszłych wersji systemu.

2.7 Istniejące systemy weryfikacji prac dyplomowych

Na rynku dostępnych jest kilka komercyjnych systemów weryfikacji jakości prac dyplomowych. Systemy te skupiają się jednak głównie na wykrywaniu plagiatów, a nie na analizie poprawności językowej. Przykładem takiego systemu jest iThenticate, szeroko stosowany przez uczelnie wyższe.

Wśród rozwiązań ukierunkowanych na analizę językową wyróżnia się platforma Grammarly, która oferuje zaawansowane możliwości korekty gramatycznej i stylistycznej dla języka angielskiego. Niestety, platforma ta nie obsługuje języka polskiego w stopniu wystarczającym do weryfikacji prac dyplomowych.

Badania nad automatyczną oceną jakości prac dyplomowych prowadzone są m.in. na kilku polskich uczelniach technicznych. Zbadałem dostępne publikacje z lat 2018 - 2024 i stwierdziłem, że największym wyzwaniem pozostaje obsługa specyficznych konwencji polskiego języka akademickiego, zwłaszcza w zakresie bezosobowych form czasownikowych i zasad użycia myślników.

Chapter 3

Metodologia

3.1 Ogólna architektura systemu

System weryfikacji poprawności językowej został zbudowany w architekturze potokowej. Dane wejściowe w postaci pliku PDF przechodzą przez kolejne moduły przetwarzające, a wynikiem działania systemu jest lista wykrytych błędów wraz z metadanymi, takimi jak pozycja błędu w dokumencie, kategoria błędu oraz komunikat opisujący naturę problemu.

Architektura systemu obejmuje następujące komponenty:

1. **Ekstraktor dokumentu** – odpowiada za ekstrakcję tekstu i metadanych strukturalnych z pliku PDF.
2. **Mapper bloków** – konwertuje surowe dane ekstraktora na ustrukturyzowany model dokumentu.
3. **Detektor języka** – identyfikuje język każdego bloku tekstu.
4. **Moduł gramatyczny** – wykrywa błędy gramatyczne i ortograficzne przy użyciu LanguageTool.
5. **Moduł skrótów** – weryfikuje poprawność definicji skrótów.
6. **Moduł list** – analizuje spójność list wyliczeniowych.
7. **Moduł form osobowych** – wykrywa użycie nieodpowiednich form czasownikowych.
8. **Moduł separatora dziesiętnego** – wykrywa błędne użycie separatora dziesiętnego.
9. **Moduł interpunkcji myślnikowej** – analizuje poprawność użycia myślników i

półpauz.

Architektura potokowa umożliwia niezależne testowanie i rozwijanie poszczególnych modułów. Każdy moduł przyjmuje ten sam ustrukturyzowany model dokumentu jako dane wejściowe i zwraca listę obiektów błędów. Wyniki ze wszystkich modułów są następnie scalane i deduplikowane przed przekazaniem do warstwy prezentacji. Uważam, że taka architektura jest optymalna z punktu widzenia rozszerzalności systemu o nowe kategorie błędów.

Komunikacja między modułami odbywa się za pośrednictwem API zdefiniowanego w module `models.py`. Każde wywołanie modułu analitycznego zwraca obiekt `ModuleResult` zawierający nazwę modułu, status wykonania, listę komentarzy diagnostycznych oraz słownik ze szczegółowymi wynikami analizy.

3.2 Moduł detekcji błędów ortograficznych

Moduł detekcji błędów ortograficznych oparty jest na bibliotece LanguageTool. Dla każdego bloku tekstowego wywoływana jest metoda `check()`, która zwraca listę dopasowań (ang. matches) zawierającą informacje o pozycji błędu, jego kategorii, komunikacie diagnostycznym oraz liście sugerowanych poprawek.

Szczególnym wyzwaniem jest odróżnienie rzeczywistych błędów ortograficznych od słów poprawnych, lecz nieznanych słownikowi LanguageTool. Dotyczy to przede wszystkim: terminologii specjalistycznej (np. nazwy bibliotek programistycznych, terminy z dziedziny informatyki), nazw własnych (nazw osób, instytucji, miejsc) oraz wyrazów obcojęzycznych. W celu ograniczenia liczby fałszywych alarmów zastosowano następujące mechanizmy filtrowania: lemmatyzację z wykorzystaniem spaCy, sprawdzanie kontekstu (czy wyraz jest kursywą lub pogrubieniem), sprawdzanie, czy wyraz pojawia się w tekście co najmniej trzy razy (co sugeruje celowe użycie terminu specjalistycznego).

Kluczowym elementem jest mechanizm sprawdzania nazw własnych. Na początku analizy z dokumentu wyodrębniane są wszystkie encje nazwane za pomocą modelu NLP. Zbiór ten jest następnie używany jako lista wyjątków przy filtrowaniu wyników LanguageTool. Dzięki temu wyrazy takie jak „Python”, „spaCy” czy „Kowalski” nie są flagowane jako błędy ortograficzne, nawet jeśli nie występują w słowniku LanguageTool.

Wyniki analizy ortograficznej są szczegółowo omówione w rozdziale 5. Precyzja modułu dla języka polskiego wynosi 0.83, co spełnia kryterium skuteczności przyjęte dla tego milestonu.

3.3 Moduł weryfikacji definicji skrótów

Moduł weryfikacji definicji skrótów analizuje dokument w poszukiwaniu: skrótów używanych bez uprzedniej definicji; skrótów definiowanych w nieprawidłowej kolejności (skrót przed rozwinięciem); oraz skrótów, których definicja pojawia się w tekście, lecz po wcześniejszym użyciu skrótu bez definicji.

Algorytm działania modułu jest następujący: w pierwszej kolejności parsowany jest spis skrótów dokumentu, o ile taki istnieje. Następnie skanowany jest tekst główny dokumentu od początku do końca. Każde wystąpienie ciągu pisanego wyłącznie wielkimi literami o długości 2–7 znaków traktowane jest jako potencjalny skrót. Dla każdego takiego ciągu sprawdzane jest, czy:

1. Skrót jest zdefiniowany w spisie skrótów na początku dokumentu;
2. Skrót jest zdefiniowany w tekście przed lub przy pierwszym użyciu w formacie: „[Pełna nazwa
3. Skrót pojawia się w formacie odwróconym: „([SKRÓT]) [Pełna nazwa]” – co jest traktowane jako błąd.

Jeśli żaden z powyższych warunków nie jest spełniony, skrót jest oznaczany jako błąd ACRONYM_DEF. W niniejszym rozdziale celowo użyto skrótu SVM bez definicji, natomiast poprawna definicja w tekście pojawia się poniżej: *Support Vector Machine (SVM)* jest algorytmem uczenia maszynowego stosowanym do klasyfikacji i regresji.

SVM znalazł zastosowanie w zadaniach NLP takich jak klasyfikacja tekstu, wykrywanie sentymentu oraz etykietowanie sekwencji. Zaletą SVM jest wysoka skuteczność przy stosunkowo małych zbiorach danych treningowych, co czyni go atrakcyjnym wyborem w scenariuszach, gdzie dostęp do dużych anotowanych korpusów jest ograniczony.

3.4 Moduł analizy spójności list

Moduł analizy spójności list wyliczeniowych implementuje algorytm opisany w sekcji 2.4. Dla każdego bloku listowego wyodrębnionego z dokumentu wykonywane są następujące operacje: klasyfikacja stylu listy (zdaniowy vs. frazowy), identyfikacja elementów niezgodnych z dominującym stylem oraz generowanie komunikatów błędów dla niezgodnych elementów.

Klasyfikacja stylu listy opiera się na analizie dominującego wzorca wśród wszystkich elementów. Jeśli więcej niż połowa elementów zaczyna się od wielkiej litery i kończy się kropką, lista klasyfikowana jest jako zdaniowa. W przeciwnym razie klasyfikowana jest jako frazowa. Elementy odbiegające od klasyfikacji są oznaczane jako błędy LIST_COHERENCE.

Poniżej przedstawiono kolejny przykład niespójnej listy – tym razem mieszającej styl zdaniowy i frazowy (celowe dane testowe):

- Ekstrakcja tekstu z pliku PDF.
- Weryfikacja bloków listowych.
- Analiza morfologiczna i syntaktyczna.
- oznaczanie błędów w strukturze dokumentu;

Moduł uwzględnia wyjątki dla elementów list będących definicjami (np. „ NLP – Przetwarzanie Języka Naturalnego”) oraz elementów zaczynających się od nazwy własnej lub skrótu. Obsługa tych wyjątków jest kluczowa dla utrzymania niskiego poziomu fałszywych alarmów.

3.5 Moduł detekcji form osobowych

Zadanie detekcji form osobowych polega na identyfikacji użycia pierwszej lub drugiej osoby czasownika w tekście akademickim. W polskich pracach dyplomowych obowiązuje zasada stosowania form bezosobowych (np. „przeprowadzono”, „zaproponowano”, „wykazano”) lub strony biernej (np. „zostało zbadane”).

Moduł wykorzystuje bibliotekę spaCy do analizy morfosyntaktycznej tekstu. Dla każdego tokenu będącego czasownikiem sprawdzane są cechy morfologiczne Person (osoba gramatyczna). Jeśli wartość Person wynosi pierwsza lub druga osoba, token jest oznaczany jako błąd PERSONAL_FORM.

Szczególnym przypadkiem są cytaty bezpośrednie. Wyrazy użyte wewnątrz cudzysłowu polskiego („. . . ”) lub angielskiego (“. . . ”) nie powinny być flagowane, nawet jeśli zawierają formy osobowe. Na przykład: jak podał autor (2023), „Przeprowadziłem szereg eksperymentów potwierdzających skuteczność metody” – wyrazy w cudzysłowie nie są oznaczane jako błąd.

3.6 Moduł separatora dziesiętnego

W języku polskim jako separator dziesiętny stosuje się przecinek, na przykład: wartość 3,14 jest poprawna, natomiast wartość 3.14 (z kropką) jest błędem. Moduł separatora dziesiętnego wykrywa wszystkie liczby z ewentualnie błędnym separatorem, a następnie filtruje fałszywe alarmy, takie jak odwołania do rozdziałów (np. rozdział 2.1), numery wersji (np. wersja 3.1.4) czy cyfry w zakresach dat.

Separatory dziesiętne w zakresach dat (np. 2019–2023) nie są flagowane. Natomiast

separatory w wyrażeniach matematycznych i statystycznych (np. wartość 0.92 zamiast 0,92) są oznaczane jako błędy. W języku angielskim sytuacja jest odwrotna: poprawnym separatorem jest kropka, a przecinek używany jako separator dziesiętny (np. wartość 2,5 zamiast 2.5) jest błędem w tekście anglojęzycznym.

3.7 Moduł interpunkcji myślnikowej

Interpunkcja myślnikowa jest źródłem wielu błędów w polskich pracach dyplomowych. W języku polskim rozróżnia się trzy znaki: dywiz (łącznik, -, Unicode U+002D), półpauzę (en-dash, –, Unicode U+2013) i pauzę (em-dash, —, Unicode U+2014). Każdy z nich ma odrębne zastosowanie, a ich mylenie jest poważnym błędem edytorskim.

Dywiz stosuje się wyłącznie do łączenia wyrazów złożonych, takich jak „naukowo-techniczny", „biało-czerwony" czy „Kowalski-Wiśniewski". Półpauza stosowana jest między liczbami w zakresach (np. „lata 2010–2020"), w roli łącznika między skrótami („NLP–ML") oraz jako myślnik we wtrąceniach, otoczony spacjami (np. „System – opisany szczegółowo w sekcji 3.2 osiąga precyzję powyżej 0,85"). Pauza (em-dash) jest rzadziej stosowana w języku polskim, lecz pojawia się w tekstach, w których autor zdecydował się na ten wariant typograficzny.

Najczęstsze błędy myślnikowe wykrywane przez moduł to:

- użycie dywizu z otaczającymi spacjami zamiast półpauzy we wtrąceniach (słowo - słowo zamiast słowo – słowo);
- użycie dywizu bez spacji w zakresach liczb (np. 2010-2020 zamiast 2010–2020);
- niespójne użycie półpauzy i pauzy w obrębie jednego dokumentu;
- użycie spacji wokół półpauzy w zakresach liczbowych (2015 – 2020 zamiast 2015–2020).

Moduł implementuje detekcję wymienionych wzorców za pomocą zestawu wyrażeń regularnych stosowanych do każdego bloku tekstowego dokumentu. Szczególna ostrożność jest wymagana przy filtrowaniu odwołań do sekcji, rozdziałów i rysunków, które mogą zawierać cyfry z separatorem kropkowym (np. sekcja 3.2) – takie wyrażenia nie powinny być flagowane jako błędy separatora dziesiętnego.

3.8 Obsługa dokumentów dwujęzycznych

Prace dyplomowe przygotowywane na polskich uczelniach technicznych zawierają często fragmenty anglojęzyczne: streszczenie w języku angielskim (wymagane przez wiele wydziałów), cytaty z anglojęzycznej literatury, angielskie nazwy technologiczne oraz

fragmenty kodu z komentarzami w języku angielskim. System musi poprawnie obsługiwać przejścia między językami, aby nie generować fałszywych alarmów dla poprawnych wyrażeń anglojęzycznych w polskim tekście.

Detekcja języka realizowana jest na poziomie bloku tekstowego przez bibliotekę. Każdy blok akapitu jest klasyfikowany jako polski lub angielski na podstawie rozkładu n znakowych charakterystycznych dla danego języka. Dla bloków o niejednoznacznej klasyfikacji (np. zawierających mieszankę terminów polskich i angielskich) stosowany jest język dominujący w dokumencie.

Szczególną uwagę poświęcono obsłudze słów angielskich w polskim tekście. Gdy moduł gramatyczny (LanguageTool-PL) wykryje potencjalny błąd ortograficzny, słowo to jest poddawane dodatkowej weryfikacji przez LanguageTool-EN. Jeśli angielski korektor uzna słowo za poprawne w języku angielskim, błąd jest anulowany – słowo jest traktowane jako celowe użycie terminu anglojęzycznego, a nie jako literówka. Mechanizm ten skutecznie eliminuje fałszywe alarmy dla terminów takich jak „embedding", „parsing", „benchmark" czy „pipeline", które są powszechnie stosowane w polskojęzycznych pracach z dziedziny informatyki.

Dla modułu form osobowych przeprowadza się analogiczną weryfikację: zdania w blokach zaklasyfikowanych jako angielskie podlegają regułom języka angielskiego (np. „we evaluated" jest flagowane), natomiast w blokach polskich stosowane są reguły języka polskiego. Bloki streszczenia anglojęzycznego są zatem analizowane niezależnie od polskiego tekstu głównego, z właściwymi dla angielskiego regułami.

3.9 Parametryzacja i konfiguracja modułów

System jest zaprojektowany jako konfigurowalny – każdy moduł może być włączony lub wyłączony niezależnie, a część modułów posiada parametry kontrolujące agresywność detekcji.

Najważniejsze parametry konfiguracyjne to: próg liczności dla filtrowania wyrazów wielokrotnych (domyślnie: 3 wystąpienia), lista własnych skrótów dodatkowych nieflagowanych przez moduł skrótów, tryb ścisły separatora dziesiętnego (domyślnie wyłączony – w trybie ścisłym nawet odwołania do podsekcji, takie jak „sekcja 3.14", są flagowane, o ile liczba po kropce nie ma formatu numeracji rozdziałów).

Mechanizm białych list umożliwia użytkownikowi dostarczenie listy wyrazów specjalistycznych, nazw własnych lub skrótów, które nie powinny być flagowane. Biała lista jest sprawdzana na etapie filtrowania wyników LanguageTool, przed ostateczną agregacją błędów. Dzięki temu możliwe jest dostosowanie systemu do specyfiki konkretnej dziedziny nauki (np. bioinformatyka, prawo, medycyna), gdzie terminy specjalistyczne

mogą być nieznane ogólnym słownikom morphologicznym.

Dokumentacja parametrów konfiguracyjnych jest dostępna w repozytorium projektu w formie schematu JSON, co umożliwia walidację pliku konfiguracyjnego przed uruchomieniem systemu. Przykładowy plik konfiguracyjny dla typowej polskiej pracy inżynierskiej jest dostarczany wraz z biblioteką jako punkt wyjścia dla użytkowników.

Chapter 4

Implementacja

4.1 Środowisko technologiczne

System został zaimplementowany w języku Python Wybór tego języka podyktowany jest dostępnością bogatego ekosystemu bibliotek NLP, w tym spaCy, LanguageTool oraz biblioteki do ekstrakcji tekstu z plików PDF. Do zarządzania środowiskiem wirtualnym użyto narzędzia Poetry, które zapewnia deterministyczne zarządzanie zależnościami.

Sprzętowe wymagania systemowe obejmują: minimum 8 GB pamięci RAM (duże modele spaCy wymagają 4–6 GB), procesor o taktowaniu minimum 2,0 GHz, dysk SSD o pojemności minimum 10 GB (modele językowe zajmują od 500 MB do 2 GB). Opcjonalnie możliwe jest wykorzystanie karty GPU, co przyspiesza przetwarzanie dużych dokumentów, lecz nie jest wymagane do poprawnego działania systemu.

Wdrożenie systemu w środowisku produkcyjnym realizowane jest za pomocą kontenerów Docker i procesów CI/CD. Każda zmiana w kodzie źródłowym wyzwala automatyczny potok budowania i testowania, który weryfikuje poprawność kodu za pomocą linterów (flake8, mypy) oraz zestawu testów jednostkowych i integracyjnych.

Application Programming Interface (API) systemu zostało zaprojektowane zgodnie ze standardem REST. Klient przesyła plik PDF poprzez żądanie HTTP do punktu końcowego `/analyze`, a system zwraca odpowiedź w formacie JSON zawierającą listę wykrytych błędów wraz z metadanymi. Każdy błąd opisany jest przez: kategorię, komunikat, pozycję w dokumencie (numer strony, indeksy słów) oraz treść błędnego fragmentu.

4.2 Ekstraktor bloków PDF

Ekstrakcja tekstu z pliku PDF jest pierwszym i jednym z najważniejszych etapów przetwarzania. System sytemu – tutaj celowy błąd literówki (powinno być „systemu”) – używa biblioteki PyMuPDF (fitz) do ekstrakcji surowego tekstu wraz z informacjami o pozycji każdego słowa na stronie.

Kluczowym wyzwaniem jest prawidłowe rozpoznawanie logicznych bloków dokumentu: akapitów, nagłówków, list wyliczeniowych, tabel i opisów skrótów. Każdy blok jest klasyfikowany na podstawie jego właściwości typograficznych (rozmiar czcionki, pogrubienie, kursywa, wcięcia).

Bloki listowe są rozpoznawane na podstawie obecności markerów list (punktor, cyfra z nawiasem, litera z nawiasem). Każdy element listy jest zapisywany jako oddzielny obiekt ze wskaźnikami do oryginalnych słów w dokumencie, co umożliwia precyzyjne lokalizowanie błędów na stronie.

Szczególnym przypadkiem są nagłówki, które w pliku PDF mogą mieć różne rozmiary czcionki i grubości. System klasyfikuje nagłówki na podstawie względnego rozmiaru czcionki w stosunku do rozmiaru czcionki tekstu podstawowego. Nagłówki pierwszego poziomu (rodziały) mają zwykle rozmiar czcionki co najmniej 1,5 razy większy niż tekst podstawowy.

Wyodrębnianie bloków z sekcji „Spis skrótów” wymaga specjalnej obsługi, ponieważ sekcja ta ma unikalną strukturę tabelaryczną. System próbuje rozpoznać tę sekcję na podstawie słów kluczowych takich jak „Spis skrótów”, „Wykaz skrótów” lub „List of abbreviations” w nagłówku bloku.

4.3 Model danych błędów

Każdy wykryty błąd jest reprezentowany jako obiekt klasy `Error_type`, zdefiniowanej w module `linguistics_types.py`. Klasa ta jest dataclassą Pythona zawierającą następujące pola:

- `content` – zawartość tekstu oznaczonego jako błąd;
- `category` – kategoria błędu;
- `message` – komunikat opisujący charakter błędu;
- `offset` – indeks znaku początku błędu w obrębie bloku;
- `error_length` – długość błędnego fragmentu w znakach;
- `block_id` – identyfikator bloku tekstowego zawierającego błąd;

- `page_start` – numer strony, na której zaczyna się błąd;
- `page_end` – numer strony, na której kończy się błąd;
- `word_idxs` – lista indeksów słów wchodzących w skład błędu.

Pole `word_idxs` jest szczególnie istotne dla interfejsu użytkownika, który musi precyzyjnie podświetlić błędny fragment w wyświetlanym dokumencie. Indeksy słów są globalne w obrębie dokumentu (nie lokalnie w obrębie strony), co ułatwia przeszukiwanie i filtrowanie błędów.

Model danych błędów jest celowo minimalistyczny – nie zawiera na przykład listy sugerowanych poprawek (dostępnej z LanguageTool). Decyzja ta była podyktowana chęcią utrzymania prostoty modelu i uniknięcia fałszywego wrażenia, że system automatycznie zna poprawną formę. Sugestie poprawek mogą być dodane w przyszłości jako opcjonalne pole, lecz nie powinny być wymagane przez interfejs.

Wyniki z wszystkich modułów są agregowane do wspólnej listy obiektów `Error_type`, która jest następnie serializowana do formatu JSON za pomocą funkcji `extract_errors_to_json()`. Plik JSON jest głównym formatem wyjściowym systemu; warstwa prezentacji (interfejs webowy lub CLI) wczytuje ten plik i renderuje błędy w odpowiednim formacie dla użytkownika.

4.4 Integracja modułów analitycznych

Integracja modułów analitycznych realizowana jest przez klasę `LinguisticsService`, która koordynuje wywołania poszczególnych modułów i scala ich wyniki. Klasa ta implementuje wzorzec projektowy Facade, ukrywając złożoność potoku przetwarzania przed warstwą kliencką.

Wywołania modułów są realizowane sekwencyjnie, w ustalonej kolejności. Kolejność ta nie jest przypadkowa – moduł skrótów musi być wywołany po module ekstrakcji nazw własnych (NER), ponieważ niektóre skróty mogą być rozwinięciami nazw własnych. Analogicznie, moduł wyjątków musi być wywołany po module gramatycznym, ponieważ filtruje jego wyniki.

W przyszłości planuje się przebudowę integracji na architekturę opartą na kolejkach komunikatów, co umożliwiłoby równoległe wykonywanie niezależnych modułów i skrócenie czasu analizy. Szacuje się, że taka przebudowa mogłaby skrócić czas analizy typowej pracy dyplomowej (80–120 stron) z obecnych 45–60 sekund do 15–20 sekund.

4.5 Mechanizmy filtrowania fałszywych alarmów

Ograniczenie liczby fałszywych alarmów jest kluczowym aspektem jakości systemu. Na podstawie wstępnej analizy zidentyfikowano kilka głównych źródeł fałszywych alarmów:

- wyrazy obcojęzyczne (głównie angielskie terminy techniczne) flagowane jako błędy ortograficzne przez LanguageTool;
- nazwy własne (nazwy bibliotek, narzędzi, firm) nieznane słownikowi;
- liczby dziesiętne w odwołaniach do rozdziałów i numerach wersji flagowane przez moduł separatora dziesiętnego;
- cytaty bezpośrednie zawierające formy osobowe czasownika.

Dla każdego źródła fałszywych alarmów zaimplementowano dedykowany mechanizm filtrowania. Efektywność tych mechanizmów jest mierzona jako liczba fałszywych alarmów na stronę tekstu – zgodnie z wymaganiami milestonu, dopuszczalne jest co najwyżej jeden fałszywy alarm na stronę.

Poniżej przedstawiono przykład listy z mieszanymi zakończeniami, dotyczącej mechanizmów filtrowania:

- filtrowanie wyrazów obcojęzycznych,
- wykrywanie nazw własnych przez moduł NER,
- obsługa cudzysłowów i cytatów.
- wykluczanie odwołań do rozdziałów.

Innym problemem jest mieszanie w jednym akapicie myślników em-dash (—) i en-dash (–): takie użycie, gdy raz stosuje się myślnik em — a innym razem półpauzę — (niekonsekwencja em/en-dash) jest traktowane jako błąd interpunkcyjny.

4.6 Projekt i dokumentacja REST API

System udostępnia funkcjonalności analityczne przez interfejs REST API, co umożliwia integrację z innymi narzędziami i platformami edukacyjnymi. Projekt API opiera się na standardzie OpenAPI i jest dokumentowany przy użyciu biblioteki FastAPI, która automatycznie generuje interaktywną dokumentację Swagger UI pod adresem `/docs`.

Czas odpowiedzi punktu końcowego `/analyze` dla typowego dokumentu dyplomowego (50–80 stron) wynosi 25–55 sekund. Ze względu na długi czas przetwarzania, API implementuje wzorzec przetwarzania asynchronicznego: klient przesyła dokument i otrzymuje identyfikator zadania, a następnie odpytuje punkt końcowy `GET /task/{id}`

w celu sprawdzenia statusu i pobrania wyników po zakończeniu analizy.

Bezpieczeństwo API opiera się na tokenach JWT (ang. JSON Web Token). Każde żądanie musi zawierać nagłówek `Authorization: Bearer {token}`. Tokeny są wydawane przez punkt końcowy `POST /auth/token` po podaniu poprawnych danych uwierzytelniających. Czas ważności tokenu wynosi 8 godzin, co odpowiada typowej sesji roboczej użytkownika.

Dokumentacja API w formacie OpenAPI jest dostępna w repozytorium projektu jako plik `openapi.yaml`. Umożliwia ona automatyczne generowanie klientów API w dowolnym języku programowania przy użyciu narzędzia openapi-generator.

4.7 Testowanie jednostkowe i integracyjne

System posiada rozbudowany zestaw testów jednostkowych i integracyjnych napisanych w frameworku pytest. Łącznie zestaw testów obejmuje ponad 350 przypadków testowych, z czego 280 to testy jednostkowe poszczególnych modułów, a 70 to testy integracyjne sprawdzające współpracę modułów w ramach potoku przetwarzania.

Testy jednostkowe dla modułu separatora dziesiętnego obejmują: przypadki poprawne (np. „0,5”, „3,14” w tekście polskim, „0.5”, „3.14” w tekście angielskim), przypadki błędne (np. „0.5” w tekście polskim, „0,5” w tekście angielskim) oraz przypadki graniczne, które nie powinny być flagowane (odwołania do rozdziałów, numery wersji, zakresy dat).

Szczególną grupę stanowią testy regresyjne, które zostały opracowane na podstawie rzeczywistych fałszywych alarmów zgłoszonych przez użytkowników beta-testujących system. Każdy zgłoszony fałszywy alarm jest reprodukowany jako przypadek testowy i dodawany do zestawu, co zapobiega regresji w przyszłych wersjach. Aktualnie zestaw testów regresyjnych obejmuje 47 przypadków dla modułu ortograficznego, 31 dla modułu separatora dziesiętnego i 28 dla modułu skrótów.

Pokrycie kodu testami (ang. code coverage) wynosi 87% dla całego systemu, z najwyższym pokryciem w modułach detekcji błędów (94–98%) i nieco niższym w modułach ekstrakcji PDF (72%), gdzie niektóre ścieżki kodu zależą od specyficznych cech wejściowego pliku PDF trudnych do odtworzenia w testach jednostkowych.

Testy integracyjne używają zestawu 15 syntetycznych dokumentów PDF opracowanych na potrzeby weryfikacji systemu, analogicznych do niniejszego dokumentu. Każdy dokument zawiera znane błędy wraz z oczekiwanymi wynikami w formacie JSON. Wynik uruchomienia testu integracyjnego jest porównywany z oczekiwanym za pomocą modułu `evaluation.py`, który oblicza precyzję, czułość i miarę F1.

4.8 Wydajność i optymalizacja

Wydajność systemu jest istotnym czynnikiem przy jego wdrożeniu w środowiskach produkcyjnych, gdzie może być konieczne przetwarzanie wielu dokumentów jednocześnie. Zmierzono czas przetwarzania dla dokumentów o różnej liczbie stron, uzyskując następujące wyniki:

- dokument 20-stronicowy: 12.3 s (śr. z 10 pomiarów);
- dokument 50-stronicowy: 28.7 s (śr. z 10 pomiarów);
- dokument 100-stronicowy: 54.2 s (śr. z 10 pomiarów).

Dominującym wąskim gardłem (ok. 65% czasu) jest inicjalizacja modeli spaCy, która odbywa się raz przy starcie serwisu i nie wpływa na czas przetwarzania kolejnych dokumentów. Samo przetwarzanie tekstu przez modele NLP zajmuje ok. 25% całkowitego czasu, a operacje wejścia/wyjścia (ekstrakcja PDF, zapis wyników) – ok. 10%.

W celu poprawy wydajności zaimplementowano mechanizm buforowania wyników lemmatyzacji przy użyciu dekoratora `@functools.cache`. Powtarzające się wyrazy (co jest szczególnie częste w przypadku terminologii specjalistycznej) nie wymagają ponownego wywołania modelu NLP, co redukuje łączny czas przetwarzania o ok. 18% dla typowej pracy dyplomowej.

Chapter 5

Eksperymenty i wyniki

5.1 Dane testowe i metodologia ewaluacji

Ewaluacja systemu została przeprowadzona na korpusie 50 prac dyplomowych: 30 w języku polskim i 20 w języku angielskim. Prace te zostały poddane ręcznej anotacji przez dwóch niezależnych językoznawców, a następnie dokonano arbitrażu w przypadku rozbieżności między anotatorami. Łącznie korpus zawiera 4200 błędów, z czego 2850 w pracach polskich i 1350 w pracach anglojęzycznych.

Podział błędów w korpusie polskim jest następujący: błędy ortograficzne i literówki stanowią 34% wszystkich błędów (970 przypadków), błędy dotyczące skrótów 18% (510 przypadków), niespójności list 15% (427 przypadków), błędy form osobowych 12% (341 przypadków), błędy separatora dziesiętnego 11% (312 przypadków) i błędy interpunkcji myślnikowej 10% (290 przypadków).

Metodologia ewaluacji opiera się na metrykach precyzji, czułości i miary F1. Stwierdziłem, że klasyczne podejście exact-match (dokładne dopasowanie kategorii, identyfikatora bloku i indeksów słów) jest najbardziej rzetelną metodą oceny, ponieważ eliminuje możliwość zaliczenia częściowo poprawnych dopasowań jako prawidłowych.

5.2 Szczegółowa analiza przypadków granicznych

Analiza przypadków granicznych jest kluczowa dla oceny jakości systemu w warunkach rzeczywistych. Zidentyfikowano cztery główne kategorie przypadków granicznych, dla których system musiał być starannie skonfigurowany, aby nie generować fałszywych alarmów.

Odwołania do rozdziałów i sekcji. Wyrażenia takie jak „rozdział 2.1”, „sekcja 3.2.4” czy „rysunek 4.1” zawierają cyfry z kropką jako separatorem, co mogłoby być mylnie in-

terpretowane jako błędny separator dziesiętny. System poprawnie filtruje te wyrażenia dzięki analizie kontekstu: jeśli liczbie z kropką bezpośrednio poprzedza słowo z białej listy („rozdział”, „sekcja”, „rys.”, „tab.”, „rysunek”, „tabela” i ich odmiany), wyrażenie nie jest flagowane. Skuteczność filtrowania: 100% (zero fałszywych alarmów w tej kategorii).

Cytaty bezpośrednie. Wyrazy i zdania wewnątrz cudzysłowów polskich („...”) nie powinny być flagowane jako błędy, nawet jeśli zawierają formy osobowe czasownika lub wyrazy nieznane słownikowi. System wykrywa granice cytatów za pomocą wyrażeń regularnych i wyklucza zawartość cytatów z analizy modułów `PERSONAL_FORM` i `TYPO`. Dla cytatów angielskich (“...”) stosowana jest analogiczna logika. Poprawny cytat z formą osobową: „Przeprowadziłem badania” – uwaga: ten konkretny fragment *jest* oznaczony błędem, ponieważ nie jest wewnątrz cudzysłowu w źródle LaTeX.

Nazwy własne i terminy specjalistyczne. Nazwy bibliotek programistycznych (spaCy, PyMuPDF, Hunspell), nazwiska autorów (Kowalski, Nowak) oraz terminy specjalistyczne (tokenizacja, lemmatyzacja) nie powinny być flagowane jako błędy ortograficzne. System używa modułu NER spaCy do rozpoznawania nazw własnych oraz mechanizmu białej listy dla terminów specjalistycznych. Ponadto, wyrazy pisane kursywą lub pogrubieniem są automatycznie wyłączane z kontroli ortograficznej, co obsługuje typograficzną konwencję wyróżniania terminów technicznych przy pierwszym użyciu.

Bloki kodu i fragmenty techniczne. Fragmenty kodu źródłowego, URI, nazwy zmiennych i ścieżki do plików zawierają wiele wyrażeń, które byłyby flagowane jako błędy przez moduły gramatyczne. System klasyfikuje bloki tekstowe o typie `code` lub `verbatim` i wyklucza je z analizy. Bloki rozpoznawane są na podstawie czcionki monospacowanej użytej w pliku PDF.

5.3 Analiza fałszywych alarmów w podziale na strony

Jednym z kluczowych kryteriów jakości systemu jest liczba fałszywych alarmów na stronę tekstu. Zgodnie z wymaganiami milestonu, dopuszczalne jest co najwyżej jeden fałszywy alarm na stronę. Poniżej przedstawiono wyniki analizy tego wskaźnika dla każdego modułu osobno.

Moduł ortograficzny generuje średnio 0.41 fałszywego alarmu na stronę dla dokumentów polskich i 0.28 dla dokumentów angielskich. Dominującym źródłem fałszywych alarmów są angielskie terminy techniczne w polskich dokumentach (ok. 62% fałszywych alarmów tego modułu).

Moduł separatora dziesiętnego generuje średnio 0.12 fałszywego alarmu na stronę –

najniższy wskaźnik spośród wszystkich modułów. Efektywne filtrowanie odwołań do rozdziałów i numerów wersji praktycznie eliminuje fałszywe alarmy w tej kategorii.

Moduł form osobowych generuje średnio 0.19 fałszywego alarmu na stronę. Fałszywe alarmy wynikają głównie z cytatów bezpośrednich, w których automatyczna detekcja granic cudzysłowu niekiedy zawodzi przy niestandardowej typografii.

Moduł spójności list wyliczeniowych generuje średnio 0.31 fałszywego alarmu na stronę. Fałszywe alarmy dotyczą głównie list definicyjnych (format: „SKRÓT – rozwinięcie"), które system niekiedy niepoprawnie klasyfikuje jako listy frazowe.

Moduł skrótów generuje średnio 0.17 fałszywego alarmu na stronę. Fałszywe alarmy dotyczą skrótów jednostek miar i walut, które system niekiedy pomija w białej liście.

Moduł interpunkcji myślnikowej generuje średnio 0.22 fałszywego alarmu na stronę. Fałszywe alarmy dotyczą głównie zapisu numerów telefonów i kodów pocztowych zawierających dywiz.

Łącznie system generuje średnio 0.87 fałszywego alarmu na stronę (uwzględniając efekty deduplikacji błędów z różnych modułów), co jest zgodne z przyjętym kryterium jakości.

5.4 Wyniki dla języka polskiego

Wyniki ewaluacji dla języka polskiego są następujące. Moduł błędów ortograficznych (TYPO) osiągnął precyzję 0,88, czułość 0,91 i F1 równe 0,89. Moduł definicji skrótów (ACRONYM_DEF) uzyskał precyzję 0,93, czułość 0,88 i F1 równe 0,90. Moduł form osobowych (PERSONAL_FORM) osiągnął precyzję 0,85, czułość 0,87 i F1 równe 0,86. Moduł spójności list (LIST_COHERENCE) uzyskał precyzję 0,82, czułość 0,86 i F1 równe 0,84. Moduł separatora dziesiętnego osiągnął precyzję 0,94, czułość 0,90 i F1 równe 0,92. Moduł interpunkcji myślnikowej uzyskał precyzję 0,88, czułość 0,85 i F1 równe 0,86. Średnia ważona dla języka polskiego wynosi 0,88 zarówno dla precyzji, czułości, jak i miary F1.

Wszystkie moduły osiągnęły wartość F1 powyżej 0,85, a precyzja powyżej 0,80, co spełnia kryterium skuteczności zdefiniowane w wymaganiach milestonu. Najwyższe wyniki uzyskał moduł separatora dziesiętnego (F1 = 0.92), co wynika z dużej regularności wzorców błędów w tej kategorii. Najniższe wyniki uzyskał moduł spójności list (F1 = 0,84), co jest spowodowane złożonością reguł dotyczących wyjątków (definicje, nazwy własne w elementach list).

Średnia liczba fałszywych alarmów na stronę wyniosła 0.87, co jest nieznacznie powyżej wartości docelowej wynoszącej 1,0. Analiza fałszywych alarmów wskazuje, że głównym

ich źródłem są anglojęzyczne terminy techniczne (35% fałszywych alarmów) oraz nazwy własne nierozpoznane przez moduł NER (28%).

5.5 Wyniki dla języka angielskiego

The English component was evaluated on 20 theses from partner universities in the United Kingdom and the United States. The evaluation methodology was identical to that used for Polish texts. We found that decimal errors are significantly less frequent in English documents, because the period is the standard decimal separator and authors rarely use commas in that role. However, occurrences such as 2,5 (decimal comma in English text) were occasionally detected in documents written by non-native speakers.

Acronym handling also differs between Polish and English texts. In English academic writing, it is common to define an acronym at its first occurrence using the format "Full Name (ABBR)", which coincides with the Polish standard. The system correctly handles both languages.

The results for English texts are as follows. The spelling module achieved precision 0.91, recall 0.93 and F1 of 0.92. Acronym definitions reached precision 0.89, recall 0.86 and F1 of 0.87. Personal forms detection achieved precision 0.87, recall 0.84 and F1 of 0.85. List coherence reached precision 0.83, recall 0.88 and F1 of 0.85. The decimal separator module achieved precision 0.95, recall 0.91 and F1 of 0.93. Punctuation errors reached precision 0.86, recall 0.83 and F1 of 0.84. The weighted average across all categories is 0.89 precision, 0.88 recall and 0.88 F1.

The following list illustrates an incoherent English enumeration (LIST_COHERENCE test case):

- Grammar errors detected with LanguageTool
- Decimal separator violations
- personal verb form detection
- Acronym definition check

5.6 Analiza błędów i przypadki graniczne

W toku analizy wyników zidentyfikowano kilka kategorii przypadków granicznych, które mają istotny wpływ na jakość ewaluacji.

Pierwszą kategorią jest problem skrótów złożonych z cyfr i liter, takich jak „3D", „2FA" czy „IPv6". Wyrażenia takie nie są traktowane jako potencjalne skróty wymagające

definicji, ponieważ zawierają cyfry.

Drugą kategorią są skróty jednostek miar i walut, takie jak „kg”, „MHz” czy „PLN”. W normach edytorskich skróty te nie wymagają definicji, o ile są powszechnie znane. Moduł skrótów korzysta z białej listy takich wyrażeń, by nie generować fałszywych alarmów.

Trzecią kategorią są odwołania do rozdziałów, sekcji i rysunków, takie jak „rozdział 2.1” czy „rysunek 3.14”. Jak wspomniano w module separatora dziesiętnego, takie odwołania nie powinny być flagowane jako błędy separatora dziesiętnego. Weryfikacja tego przypadku granicznego na zbiorze testowym wykazała zero fałszywych alarmów (100% precyzja) w tej konkretnej kategorii przypadków.

5.7 Analiza czasu i złożoności obliczeniowej

Złożoność obliczeniowa systemu jest liniowa względem liczby słów w dokumencie dla większości modułów. Wyjątkiem jest moduł sprawdzania nazw własnych (moduł NER spaCy), który ma złożoność O(n) z wyższą stałą ze względu na użycie modelu transformerowego. Moduł weryfikacji definicji skrótów ma złożoność O(n·m), gdzie m jest liczbą unikatowych skrótów w dokumencie, lecz w praktyce m jest mała (zwykle poniżej 50), więc moduł ten nie stanowi wąskiego gardła.

Pomiary dla dokumentu 50-stronicowego (ok. 15000 słów) wykazały następujące czasy wykonania poszczególnych modułów: ekstrakcja PDF – 2,1 s, moduł NER i nazwy własne – 8,4 s, LanguageTool (moduł gramatyczny) – 11,3 s, moduł form osobowych – 4,2 s, moduł skrótów – 0,3 s, moduł list – 0,8 s, moduł separatora dziesiętnego – 0,1 s, moduł myślnikowy – 0,1 s. Łączny czas przetwarzania wynosi ok. 28 sekund, z czego LanguageTool i modele spaCy stanowią razem ok. 85% całości.

5.8 Porównanie z istniejącymi narzędziami

W celu kontekstualizacji wyników przeprowadzono porównanie systemu z dostępnymi narzędziami: samym LanguageTool (bez dodatkowych modułów) oraz komercyjną platformą Grammarly (wyłącznie dla dokumentów anglojęzycznych).

Dla dokumentów polskich LanguageTool bez dodatkowych modułów osiąga precyzję 0,71 i czułość 0,68, co daje F1 = 0,69. Niskie wartości precyzji wynikają z dużej liczby fałszywych alarmów generowanych dla terminologii specjalistycznej i nazw własnych. Proponowany system, dzięki mechanizmom filtrowania, osiąga F1 = 0,88, co stanowi poprawę o 0,19 punktu procentowego.

Grammarly, oceniany na dokumentach anglojęzycznych, osiąga precyzję 0,86 i czułość 0,81 (F1 = 0,83), przy czym platforma ta nie obsługuje kategorii definicji skrótów ani spójności list wyliczeniowych, które są specyficzne dla oceny prac dyplomowych. Proponowany system osiąga F1 = 0,88 dla dokumentów anglojęzycznych, co jest nieznacznie lepszym wynikiem, lecz z uwzględnieniem szerszego zakresu analizowanych kategorii błędów.

Warto podkreślić, że żadne z porównywanych narzędzi nie oferuje analizy statystycznej proporcji zdań biernych/bezosobowych, weryfikacji spójności list wyliczeniowych ani dedykowanego modułu do weryfikacji definicji skrótów zgodnie z polskimi normami edytorskimi. W tych wymiarach proponowany system nie ma bezpośrednich konkurentów w przestrzeni narzędzi open-source.

Chapter 6

Dyskusja

6.1 Interpretacja wyników

Uzyskane wyniki potwierdzają, że automatyczna weryfikacja poprawności językowej prac dyplomowych jest wykonalna z wysoką skutecznością. Wszystkie moduły systemu osiągają wartości F1 powyżej 0,85, co spełnia kryterium skuteczności zdefiniowane w wymaganiach milestonu. Szczególnie wysoka skuteczność modułu separatora dziesiętnego wynika z regularności wzorców błędów w tej kategorii – liczby z błędnym separatorem są stosunkowo łatwe do identyfikacji za pomocą wyrażeń regularnych, pod warunkiem poprawnego filtrowania przypadków granicznych.

Zauważyłem, że trudniejsze do detekcji są błędy spójności list i niespójna interpunkcja myślnikowa. W obu przypadkach wyniki są nieco niższe niż dla pozostałych kategorii, lecz wciąż spełniają przyjęte kryteria. Szczegółowa analiza błędnych klasyfikacji wskazuje, że głównym problemem jest niejednoznaczność definicji „spójności" – niektóre listy mieszane mają uzasadnienie merytoryczne (np. element będący cytatem może mieć inne reguły interpunkcyjne niż pozostałe elementy).

Warto odnotować, że dla zakresu dat i liczb moduł interpunkcji myślnikowej działa poprawnie. Na przykład, zapis „lata 2020–2024" (z półpauzą bez spacji) jest poprawny w języku polskim i nie jest flagowany. Natomiast zapis lata 2020 - 2024 (z dywizem i spacjami) jest błędem i jest prawidłowo identyfikowany.

Wyniki dla języka angielskiego są nieco wyższe niż dla języka polskiego, co można tłumaczyć kilkoma czynnikami: bogatsza morfologia polska generuje więcej wyjątków w regułach, angielskie modele spaCy są wytrenowane na znacznie większych korpusach, a konwencje edytorskie języka angielskiego są bardziej zunifikowane. Wskaźnik ROC (nigdy niezdefiniowany w tym dokumencie) dla klasyfikatora binarnego wyniosłby ponad 0,95, co jest dodatkowym potwierdzeniem wysokiej skuteczności systemu.

6.2 Ograniczenia systemu

Pomimo zadowalających wyników ewaluacji, system posiada szereg ograniczeń, które należy uwzględnić przy jego wdrożeniu w środowisku produkcyjnym.

Pierwszym ograniczeniem jest zależność od jakości ekstrakcji tekstu z PDF. Dokumenty o niestandardowym układzie graficznym, zawierające złożone tabele lub wiele kolumn tekstu, mogą być niepoprawnie parsowane. W takich przypadkach moduły analityczne działają na niepoprawnie wyodrębnionym tekście, co może generować zarówno fałszywe alarmy, jak i pominięcia błędów.

Drugim ograniczeniem jest brak obsługi przypisów i bibliografii. Tradycyjnie w przypisach dolnych dopuszczalne jest stosowanie skróconej formy zapisów, w tym skrótów bez definicji. System nie rozróżnia przypisów od tekstu głównego, co może generować fałszywe alarmy w tej części dokumentu.

Trzecim ograniczeniem jest obsługa tekstu matematycznego. Wzory i wyrażenia matematyczne mogą zawierać symbole, które są mylnie interpretowane jako skróty lub błędne separatory dziesiętne. Aktualnie system stara się pomijać bloki tekstowe sklasyfikowane jako wzory matematyczne, lecz mechanizm ten nie jest niezawodny.

6.3 Reprodukowalność wyników i publiczny korpus testowy

Reprodukowalność wyników jest kluczowym aspektem rzetelności badań naukowych. Korpus zawiera 50 prac dyplomowych (po uzyskaniu zgody autorów oraz zanonimizowaniu danych osobowych) wraz z plikami JSON opisującymi oczekiwane wyniki anotacji.

Reprodukcja wyników wymaga zainstalowania środowiska zgodnie z instrukcją w pliku `README.md` repozytorium oraz uruchomienia skryptu `run_evaluation.py`, który automatycznie przetwarza wszystkie dokumenty z korpusu i generuje raport wyników. Całkowity czas reprodukcji na maszynie z procesorem Intel Core i7 i pamięcią 16 GB wynosi ok. 35 minut.

Zaobserwowano, że wyniki mogą się nieznacznie różnić w zależności od wersji biblioteki spaCy i LanguageTool, ponieważ modele NLP są regularnie aktualizowane. W repozytorium zafixowano wersje wszystkich zależności w pliku `pyproject.toml`, aby zapewnić powtarzalność wyników. Testy z najnowszymi wersjami bibliotek (spaCy, LanguageTool) wykazały zmianę metryki o nie więcej niż $\pm 0{,}02$ w stosunku do wartości podanych w tabeli.

6.4 Kierunki dalszego rozwoju

W przyszłości planuje się rozszerzenie systemu o kilka istotnych funkcjonalności. Po pierwsze, chciałbym zbadać możliwości zastosowania modeli transformerowych do klasyfikacji błędów stylistycznych, w szczególności do detekcji redundancji i powtórzeń leksykalnych. Wstępne eksperymenty z modelem `herbert-base-cased` wskazują na obiecujące wyniki w tym zakresie.

Po drugie, planowane jest opracowanie modułu analizy struktury pracy – weryfikacji, czy praca zawiera wszystkie wymagane sekcje (wstęp, przegląd literatury, metodologię, eksperymenty, dyskusję i wnioski) we właściwej kolejności. Moduł ten byłby szczególnie użyteczny dla promotorów weryfikujących pierwszą wersję pracy.

Po trzecie, rozważana jest integracja z platformami edukacyjnymi, takimi jak Moodle czy Blackboard, co umożliwiłoby automatyczne uruchamianie analizy przy składaniu pracy przez studenta. Integracja ta wymagałaby implementacji dodatkowego modułu uwierzytelniania i zarządzania dostępem.

6.5 Implikacje dla dydaktyki akademickiej

Wdrożenie systemu automatycznej weryfikacji językowej może mieć istotny wpływ na proces dydaktyczny. Promotorzy zgłaszają, że znacząca część ich czasu poświęcanego na korektę prac dyplomowych jest zajęta wskazywaniem elementarnych błędów językowych, które student powinien był samodzielnie wyeliminować. Automatyzacja tej fazy korekty pozwoliłaby promotorowi skupić się na merytorycznej ocenie pracy.

Badania ankietowe przeprowadzone wśród 42 promotorów na Wydziale Informatyki wykazały, że 78% z nich spędza średnio 3–5 godzin na korektę językową jednej pracy magisterskiej. Przy założeniu, że system automatyczny eliminuje 60% tych błędów przed złożeniem pracy do promotora, oszczędność czasu wynosi ok. 2–3 godziny na pracę. W skali wydziału składającego 150 prac rocznie daje to oszczędność rzędu 300–450 godzin pracy dydaktycznej rocznie.

Z perspektywy studenta, dostęp do automatycznej korekty na etapie pisania pracy ma wartość dydaktyczną wykraczającą poza samą korektę: student otrzymuje natychmiastową informację zwrotną o popełnionych błędach, co sprzyja uczeniu się i unikaniu tych samych błędów w przyszłości. Efekt ten jest szczególnie wyraźny w przypadku błędów systemowych, takich jak stosowanie form osobowych zamiast bezosobowych, gdzie jednokrotna świadoma korekta może trwale zmienić nawyki językowe piszącego.

Wdrożenie systemu jako obowiązkowego narzędzia przedskładania pracy (ang. pre-submission checker) wymagałoby ustalenia polityki uczelni dotyczącej dopuszczalnej

liczby błędów. Propozycja autorów jest taka, aby system raportował błędy, lecz nie blokował złożenia pracy – ostateczna decyzja o akceptacji zawsze powinna należeć do promotora.

Chapter 7

Wnioski

7.1 Podsumowanie wyników

W niniejszej pracy przedstawiono projekt, implementację i ewaluację systemu automatycznej weryfikacji poprawności językowej dokumentów akademickich. System obejmuje sześć modułów analitycznych: detekcję błędów ortograficznych, weryfikację definicji skrótów, analizę spójności list wyliczeniowych, detekcję form osobowych, weryfikację separatorów dziesiętnych oraz analizę interpunkcji myślnikowej.

Wyniki eksperymentów ewaluacyjnych przeprowadzonych na korpusie 50 prac dyplomowych wykazały, że system osiąga wartości F1 powyżej 0,85 dla wszystkich analizowanych kategorii błędów. Precyzja przekracza 0,80 dla wszystkich modułów, a średnia liczba fałszywych alarmów na stronę wynosi 0.87, co jest zbliżone do przyjętego kryterium 1,0. Jestem przekonany, że dalszy rozwój systemu pozwoli osiągnąć jeszcze niższą liczbę fałszywych alarmów dzięki udoskonaleniu mechanizmów filtrowania nazw własnych i terminologii specjalistycznej.

Spośród wszystkich zaimplementowanych modułów najwyższe wyniki osiąga moduł separatora dziesiętnego (F1 = 0,92), co wynika z regularności wzorców błędów. Moduł spójności list wyliczeniowych osiąga najniższe wyniki (F1 = 0,84), co jest spowodowane złożonością reguł dotyczących wyjątków. Mimo to, wszystkie moduły spełniają przyjęte kryterium skuteczności, co potwierdza osiągnięcie celu niniejszej pracy.

Porównanie z narzędziem LanguageTool użytym bez dodatkowych modułów wykazuje poprawę F1 o 0,19 punktu dla języka polskiego, co jest istotną statystycznie różnicą. Potwierdza to zasadność opracowania dedykowanego systemu z mechanizmami filtrowania fałszywych alarmów dostosowanymi do specyfiki prac dyplomowych.

Wyniki analizy statystycznej proporcji struktur zdaniowych wskazują, że prace dyplo-

mowe oceniane pozytywnie pod względem stylistycznym cechują się wyższym udziałem zdań biernych i bezosobowych niż prace oceniane negatywnie. Fakt ten potwierdza zasadność uwzględnienia analizy statystycznej jako dodatkowego wymiaru oceny jakości językowej pracy dyplomowej, wykraczającego poza samą detekcję błędów.

7.2 Wkład własny i dalsze perspektywy

Głównym wkładem pracy jest opracowanie kompleksowego systemu weryfikacji językowej uwzględniającego specyfikę polskich prac dyplomowych, w tym unikalny algorytm weryfikacji definicji skrótów z obsługą zarówno tekstu głównego, jak i spisu skrótów. Chciałbym rozszerzyć niniejszą pracę o moduł analizy stylistycznej, który oceniałby nie tylko poprawność, lecz także jakość języka akademickiego w wymiarze semantycznym i retorycznym.

System jest dostępny jako biblioteka open-source. Dokumentacja, przykłady użycia oraz zbiory danych testowych są dostępne w repozytorium projektu. Planowane jest dalsze rozwijanie systemu we współpracy z uczelniami partnerskimi w ramach projektu badawczego finansowanego ze środków NCN.

Niniejsza praca wpisuje się w rosnący nurt badań nad automatyzacją wspomagania redakcji tekstu naukowego. Wyniki wskazują, że przy odpowiednim podejściu do obsługi języka polskiego możliwe jest osiągnięcie skuteczności porównywalnej z narzędziami dla języka angielskiego, co otwiera perspektywę szerokiego zastosowania takich systemów w polskim szkolnictwie wyższym.

Planowane jest również opublikowanie wyników niniejszej pracy w formie artykułu konferencyjnego na konferencji poświęconej technologiom językowm, takich jak Poleval lub TSD (Text, Speech, Dialogue). Artykuł będzie zawierał szczegółowy opis metod oraz wyniki porównawcze z innymi systemami, wykraczające poza zakres niniejszej pracy.

7.3 Uwagi etyczne i ograniczenia prawne

Automatyczna weryfikacja tekstu rodzi pewne kwestie etyczne i prawne, które wymagają rozważenia przy wdrożeniu systemu. Po pierwsze, kwestia prywatności: prace dyplomowe zawierają dane osobowe autorów oraz potencjalnie wrażliwe informacje badawcze. Przetwarzanie tych dokumentów przez systemy zewnętrzne wymaga odpowiedniej polityki ochrony danych zgodnej z RODO.

W proponowanym systemie zadbano o to, aby przetwarzanie odbywało się lokalnie na serwerze uczelni, bez przesyłania treści dokumentów do zewnętrznych serwisów. Je-

dynym wyjątkiem jest korzystanie z zewnętrznego serwera LanguageTool, które jednak może być zastąpione lokalną instalacją biblioteki. Dokumentacja zawiera instrukcję konfiguracji systemu w trybie w pełni offline.

Po drugie, kwestia autonomii studenta: system powinien wspierać, a nie zastępować, proces uczenia się poprawnego pisania. Prezentowanie wyników w formie sugestii (a nie błędów wymagających obowiązkowej korekty) oraz towarzyszące wyjaśnienia natury błędów mają na celu edukację studenta, a nie mechaniczne generowanie poprawnych dokumentów.

Po trzecie, kwestia odpowiedzialności za fałszywe alarmy: system może wskazać jako błąd fragment tekstu, który jest poprawny w danym kontekście specjalistycznym. Odpowiedzialność za ostateczną ocenę zawsze spoczywa na autorze i promotorze – system jest narzędziem pomocniczym, nie arbiterem poprawności.

7.4 Podsumowanie końcowe

System automatycznej weryfikacji poprawności językowej prac dyplomowych opracowany w ramach niniejszej pracy stanowi istotny krok w kierunku automatyzacji procesu oceny jakości dokumentów akademickich. Osiągnięte wyniki potwierdzają wykonalność podejścia i jego przydatność praktyczną. Kod źródłowy systemu, corpus testowy i dokumentacja są dostępne publicznie, co umożliwia dalszy rozwój przez społeczność akademicką.

Wierzę, że narzędzia takie jak opisany system będą odgrywać coraz ważniejszą rolę w ekosystemie narzędzi wspierających pisanie prac naukowych. Równocześnie ważne jest, aby rozwijać je w sposób odpowiedzialny: z poszanowaniem prywatności, z zachowaniem autonomii autora i z świadomością ograniczeń automatycznej analizy językowej.